

Computação Gráfica com Love2D

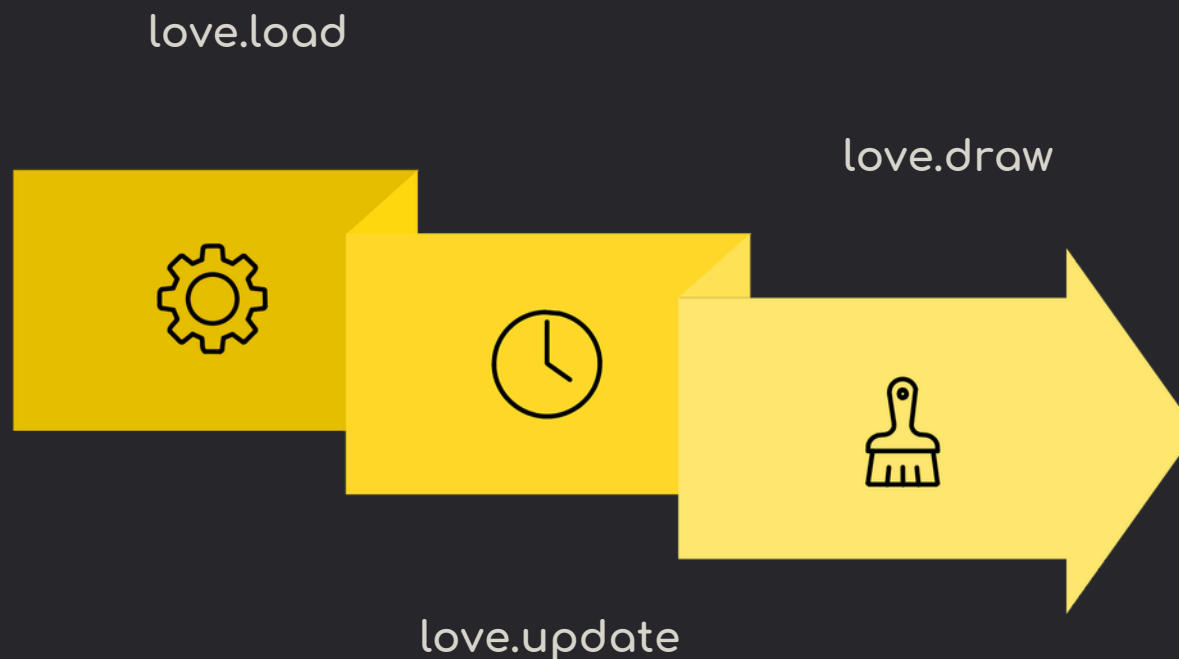
2D em Primeiro Plano, 3D com Consciência

Um seminário sobre frameworks gráficos, renderização e as escolhas certas para aprender computação gráfica na prática.



O Coração da Renderização: o Loop do Love2D

Todo frame de um jogo em Love2D passa por três etapas fixas—o **game loop** que sustenta toda a renderização.



Desenhar em Love2D é **compor a cena frame a frame** — cada callback tem um papel claro e insubstituível.

2D: O Que o Love2D Faz Muito Bem

love.graphics

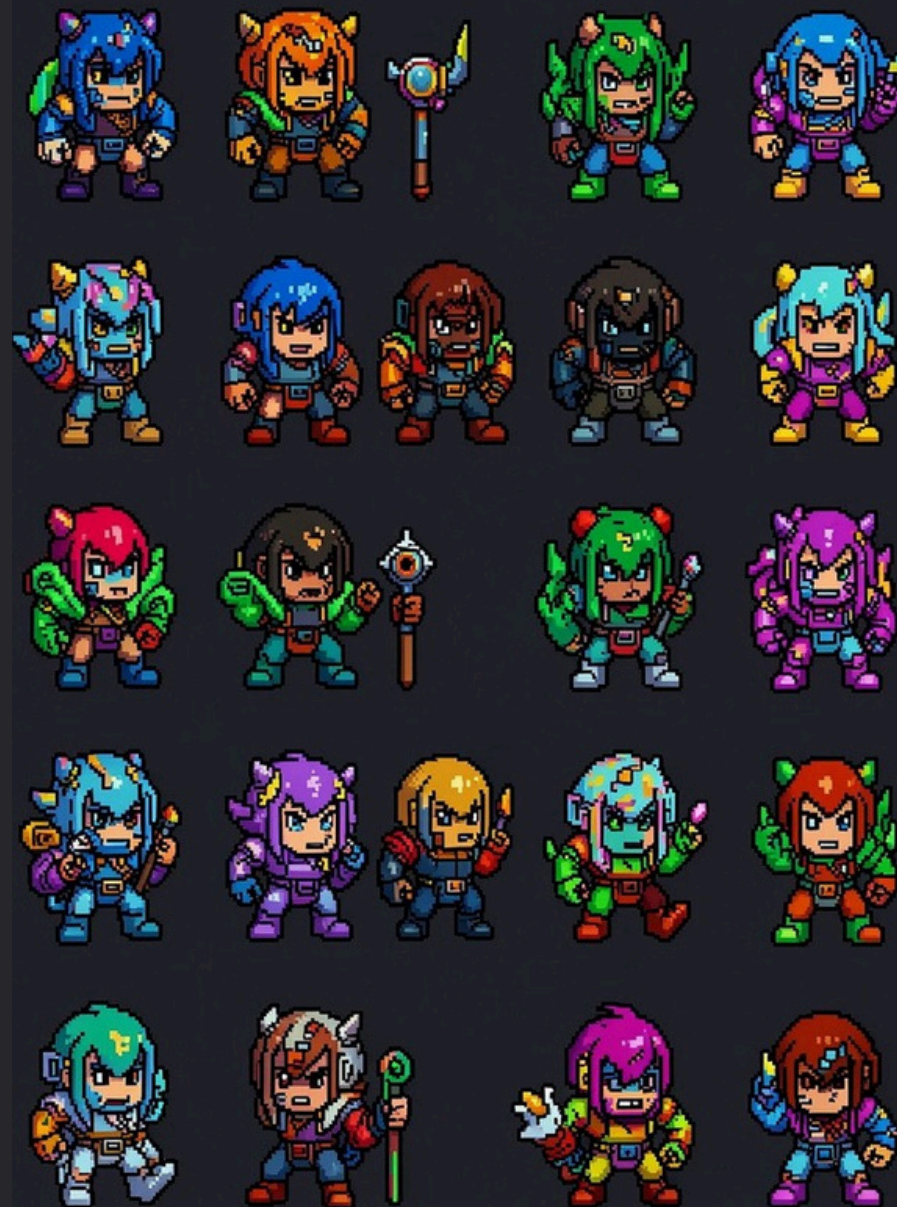
Módulo central para operações gráficas: formas, imagens, texto e transformações.

Sprites como Entidades

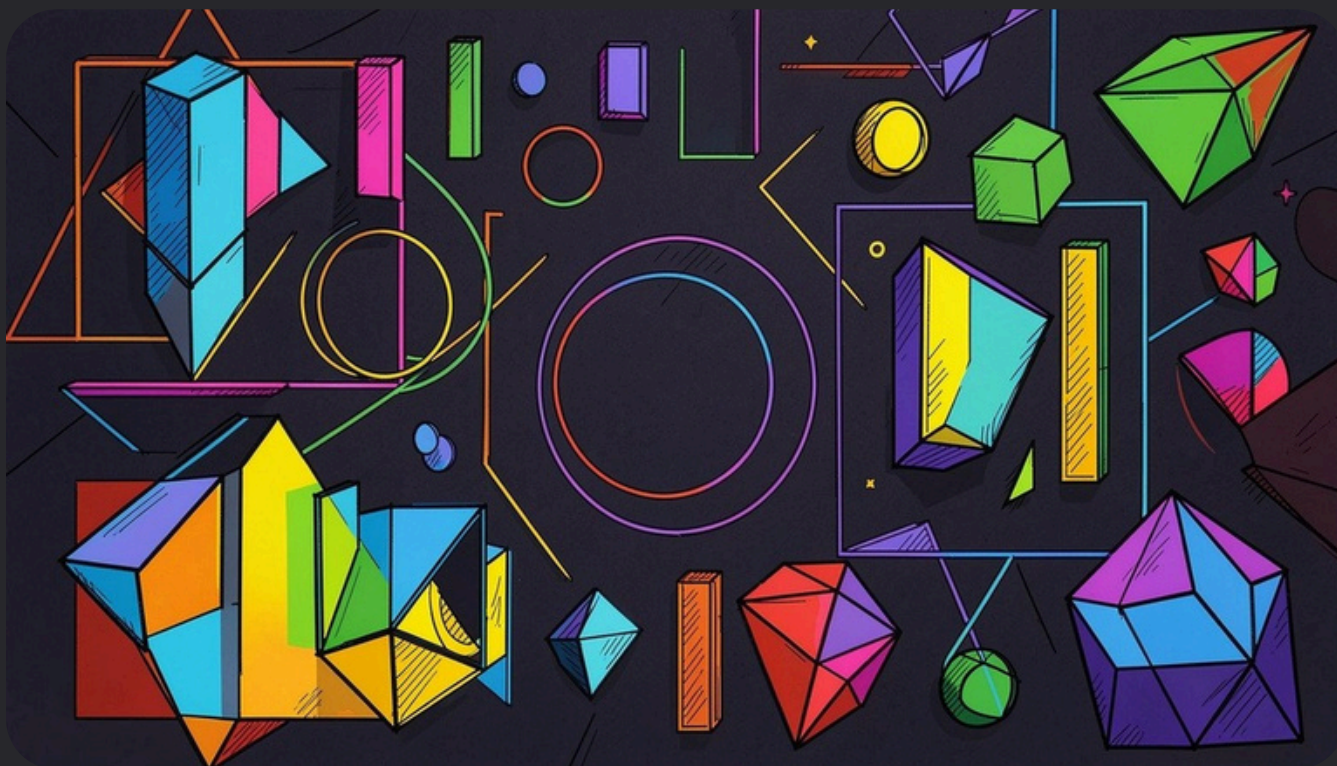
Recursos carregados viram objetos na cena — carregue uma vez, desenhe a cada frame.

Renderização por Callback

Toda chamada de desenho ocorre dentro de `love.draw`, repetida automaticamente a cada frame.



Primitivas Geométricas: Desenhar Sem "Magia"



O Love2D oferece primitivas prontas para construir cenas do zero. Cada forma é um comando direto:

love.graphics.rectangle

Retângulos com posição, tamanho e arredondamento.

love.graphics.circle

Círculos com raio e modo de preenchimento.

love.graphics.line

Linhas conectando pontos de coordenadas.

love.graphics.polygon

Polígonos arbitrários definidos por vértices.



Use `love.graphics.setColor(r, g, b,a)` para ajustar a cor antes de qualquer primitiva.

De Pixels Soltos a Cena: Coordenadas e Organização



Posição

Estado

Render

Sistema de Coordenadas 2D

Cada objeto tem **X** e **Y** na tela. O ponto (0, 0) fica no canto superior esquerdo.

Separar Estado de Renderização

Atualize posições e velocidades em `love.update`.
Desenhe em `love.draw`. Misturar os dois gera bugs difíceis de rastrear.

Interação Gráfica 2D: Input Vira Comportamento Visual



love.keyboard

Detecta teclas pressionadas e liberadas. Altera estados que afetam sprites e animações.



love.mouse

Captura posição, cliques e movimento. Ideal para menus, mira e interfaces.

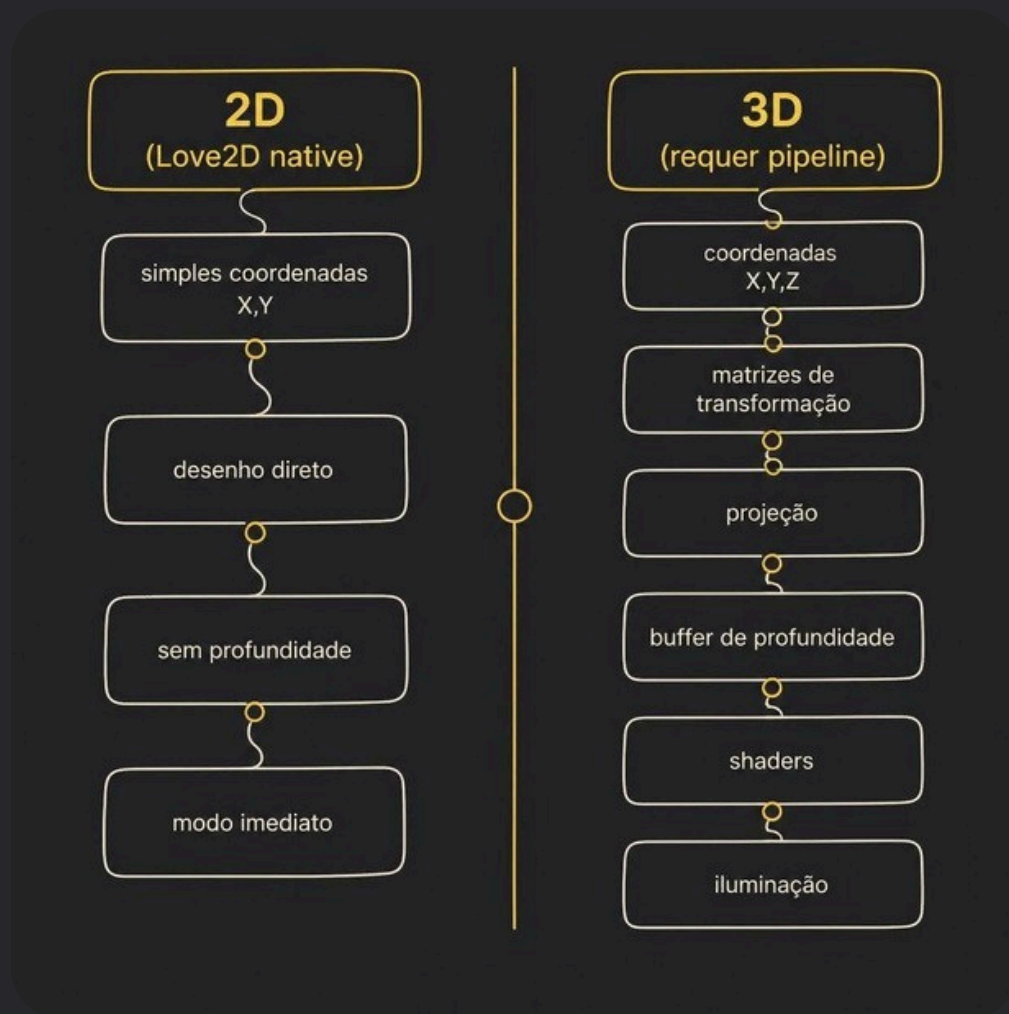


Design por Callbacks

Input e renderização vivem em callbacks separados — **controle nunca se mistura com desenho.**



A Transição para 3D: O Que Muda de Verdade



O Limite do Love2D

O Love2D não suporta 3D nativo. Renderizar em 3D exige um **pipeline gráfico completo**: transformações, projeção, iluminação e shaders — tudo ausente no modelo 2D imediato do Love2D.

- "3D em Love2D" significa simulação ou projeção — não renderização 3D real.

Como Ensinar 3D Mesmo Usando Love2D

Sem suporte nativo, ainda é possível **ensinar os conceitos** de 3D com duas estratégias práticas:

Estratégia A — Projeção Matemática

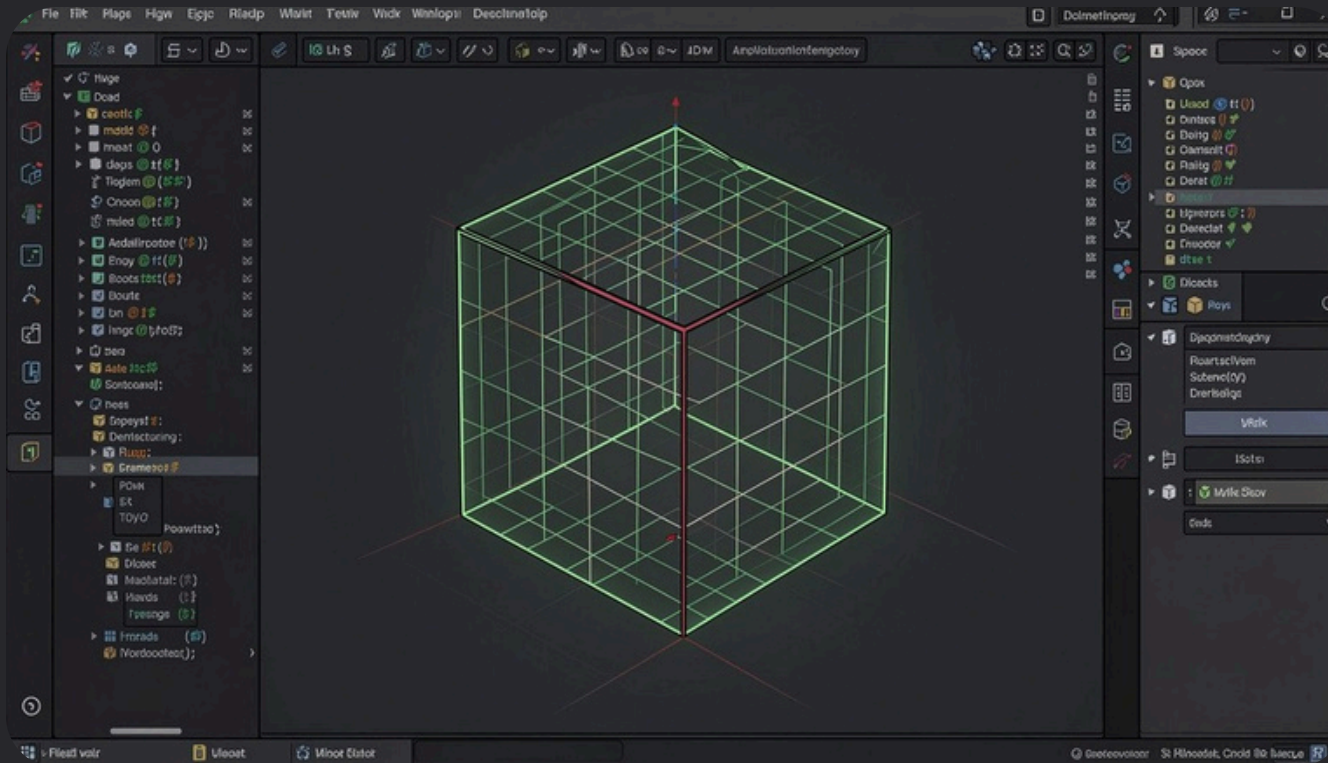
Transforme coordenadas 3D (X, Y, Z) em 2D usando fórmulas de projeção perspectiva. O resultado é desenhado com primitivas 2D normais.

Estratégia B — Pseudo-3D em Camadas

Simule profundidade com técnicas como *parallax scrolling*, perspectiva forçada e sobreposição de camadas 2D — efeito visual 3D, execução 2D.

A mensagem central: **os conceitos de 3D são universais** — a execução depende do framework.

Caminhos Reais para 3D Além do Love2D



Quando o 3D Nativo é Exigido

Seo projeto requer 3D real,migre para uma stack que implemente o pipeline gráfico completo:

01

OpenGL + GLFW

Cria contexto,janelas e gerencia buffers de vértices.

02

Shaders em GLSL

Programas que rodam na GPU para transformação e iluminação.

03

Transformações Geométricas

Matrizes de modelo, visão e projeção — o coração da CG moderna.

Conclusão: O Framework Define o Caminho, Não o Aprendizado

No Love2D

Você domina fundamentos de renderização 2D: callbacks, `love.graphics` e organização de cena frame a frame.

Para 3D

Você aprende os conceitos — pipeline, projeção, shaders — e escolhe o meio correto: simulação 2D ou outra tecnologia.

A Lição Final

A cena perfeita começa no loop certo e termina na **escolha consciente do framework**.

